

Notification App – Three Tier Architecture now integrated with PostgreSQL and ADO.NET

Problem Statement

Create a console application for a company that wants to send notifications to users. The company currently supports:

- Email Notification
- SMS Notification

In the future, the system can be extended to support:

- WhatsApp Notification
- Push Notifications

The application is implemented using a Three-Tier Architecture with PostgreSQL database connectivity using ADO.NET.

Requirements

1. Presentation Layer

Create a console menu that allows the user to:

1. Add user details
2. Choose notification type
 - Email
 - SMS
3. Enter notification message
4. Send notification
5. Display all sent notifications
6. Display notification by ID
7. Update notification
8. Delete notification
9. Display all users

2. Business Logic Layer

Create business rules before sending notifications:

1. Message should not be empty
 2. Message length should be at least 5 characters
 3. Email notifications should be sent only if the user has a valid email
 4. SMS notifications should be sent only if the user has a valid phone number
 5. SMS message should not exceed 160 characters
 6. Email message should not exceed 1000 characters
 7. Sent date should be automatically generated
 8. System should select the correct notification sender dynamically
 9. Exceptions should be handled properly
-

3. Data Access Layer

Instead of storing data in collections, PostgreSQL database storage is implemented using ADO.NET and Npgsql.

The data layer supports:

- Add user
 - Retrieve user
 - Store notification
 - Retrieve all notifications
 - Retrieve notification by ID
 - Update notification
 - Delete notification
 - Retrieve all users
-

Database Design

Users Table

```
CREATE TABLE IF NOT EXISTS users
(
    user_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150),
    phone VARCHAR(20)
);
```

Name – Sai Nikitha NSR
Date – 11th May 7, 2026.

Day 13 Three Tier Architecture – Notification App
Mail – sainikinsr@gmail.com

Notifications Table

```
CREATE TABLE IF NOT EXISTS notifications
(
    notification_id SERIAL PRIMARY KEY,
    message TEXT NOT NULL,
    sent_date TIMESTAMP,
    notification_type VARCHAR(20),
    user_id INT REFERENCES users(user_id)
);
```

Three-Tier Architecture Design

1. Presentation Layer

Program.cs

Responsible for:

- Displaying menu
 - Reading user inputs
 - Displaying outputs
 - Calling business layer methods
 - Exception handling
-

2. Business Layer

NotificationService.cs

Contains all business logic:

Methods:

- SendNotification()
- ValidateUser()
- ValidateMessage()
- ApplyRules()
- NormalizeType()
- GetSender()

Responsibilities:

- Validation handling

Name – Sai Nikitha NSR
Date – 11th May 7, 2026.

Day 13 Three Tier Architecture – Notification App
Mail – sainikinsr@gmail.com

- Notification rules
- Sender selection
- Managing notification flow

NotificationExceptions.cs

Contains:

- NotificationValidationException
- NotificationProcessException

Purpose:

- Separate validation errors from processing errors
- Centralized business exception handling

3. Data Access Layer

NotificationRepository.cs

Responsible for:

- PostgreSQL connectivity
- Executing SQL queries
- CRUD operations using ADO.NET
- Retrieving and storing data

Methods:

- AddUser()
- GetUser()
- Add()
- GetAll()
- GetByIndex()
- Update()
- Delete()
- GetAllUsers()

Concepts Used:

- Repository Pattern
- CRUD Operations

Name – Sai Nikitha NSR

Date – 11th May 7, 2026.

Day 13 Three Tier Architecture – Notification App

Mail – sainikinsr@gmail.com

- PostgreSQL Connectivity
 - ADO.NET
 - SQL Query Execution
 - Npgsql
-

Models

User.cs

Represents a user receiving notifications.

Properties:

- UserId
- Name
- Email
- Phone

Methods:

- IsValidEmail()
 - IsValidPhone()
 - GetShortInfo()
-

Notification.cs

Represents notification information.

Properties:

- NotId
- NotType
- Message
- SentDate
- Sender

Methods:

- GetAuditLine()
- HasContent()

Interfaces

INotificationSender.cs

Defines a common notification sending contract.

Method:

```
void Send(User user, Notification notification);
```

Implemented by:

- EmailNotificationSender
- SmsNotificationSender

Concept Demonstrated:

- Runtime Polymorphism
-

IRepository.cs

Defines repository operations.

Methods:

- Add()
- GetAll()
- GetByIndex()
- Update()
- Delete()
- AddUser()
- GetUser()
- GetAllUsers()

Concepts Demonstrated:

- Abstraction
 - Repository Pattern
-

Notification Sender Classes

EmailNotificationSender.cs

Responsible for:

- Formatting email notifications
 - Simulating email sending
-

SmsNotificationSender.cs

Responsible for:

- Formatting SMS notifications
 - Simulating SMS sending
-

Concepts Used

- Three-Tier Architecture
 - PostgreSQL Database Connectivity
 - ADO.NET
 - Repository Pattern
 - Interfaces
 - Runtime Polymorphism
 - Encapsulation
 - LINQ
 - CRUD Operations
 - Exception Handling
 - Business Rule Validation
 - Menu Driven Console Application
 - Separation of Concerns
-

LINQ Usage

The application uses LINQ for validation logic.

Name – Sai Nikitha NSR
Date – 11th May 7, 2026.

Day 13 Three Tier Architecture – Notification App
Mail – sainikinsr@gmail.com

Example:

```
digits.All(char.IsDigit)
```

Used for validating phone numbers.

Application Flow

1. User selects an option from menu
 2. Program.cs collects input
 3. NotificationService validates input
 4. Business rules are applied
 5. Correct sender object is selected
 6. Notification is sent
 7. Data is stored in PostgreSQL database
 8. Notifications can later be viewed, updated, or deleted
-

CRUD Operations Supported

Create

- Add user
- Send notification

Read

- View all notifications
- View notification by ID
- View all users

Update

- Update notification message

Delete

- Delete notification

Exception Handling

Validation Exceptions

Handled using:

- NotificationValidationException

Examples:

- Invalid email
- Invalid phone number
- Empty message
- Invalid notification type

Processing Exceptions

Handled using:

- NotificationProcessException

Examples:

- Database connectivity failures
- Unexpected runtime errors

Github Repo Link : [Click here](#)